HPC Assignment – 1

Name: M.ABHISHEK
HT NO: 2303A51762
Batch - 11

1. Local System -

```
===== HPC Triple Loop Matrix Multiplication =====
System       : Darwin
Processor    : arm
CPU Cores    : 8
Python Version: 3.13.7
Working Dir  : /Users/ricky/Documents/BTECH/Assignments/3rd Year/6th
Sem/HPC/Code

Matrix Size  : 200 x 200

===== Performance Metrics =====
Execution Time      : 1.7134 seconds
Current Memory Usage: 1.3295 MB
Peak Memory Usage   : 1.3304 MB

===== Function Call Profiling =====
         730 function calls (724 primitive calls) in 1.712 seconds

   Ordered by: internal time
   List reduced from 155 to 10 due to restriction <10>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    1.454    1.454    1.454    1.454 2661988393.py:17(matrix_multiply)
        3    0.188    0.063    0.188    0.063 {method 'control' of 'select.kqueue' objects}
        8    0.050    0.006    0.050    0.006 socket.py:623(send)
Job completed successfully!
```

2. Google Colab CPU -

```
Execution Time: 7.5553 seconds
Current Memory Usage: 1.31 MB
Peak Memory Usage: 1.32 MB
         76 function calls (75 primitive calls) in 7.554 seconds

   Ordered by: internal time
```

```
   List reduced from 30 to 10 due to restriction <10>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        6    6.040    1.007    6.040    1.007 {built-in method time.sleep}
        1    0.847    0.847    0.847    0.847 /tmp/ipython-input-
2468726582.py:12(matrix_multiply)
        1    0.667    0.667    7.553    7.553 /tmp/ipython-input-2468726582.py:1(<cell
line: 0>)
        2    0.000    0.000    0.000    0.000 {built-in method builtins.compile}
      3/2    0.000    0.000    7.554    3.777 /usr/local/lib/python3.12/dist-
packages/IPython/core/interactiveshell.py:3512(run_code)
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        2    0.000    0.000    0.000    0.000 /usr/local/lib/python3.12/dist-
packages/IPython/core/interactiveshell.py:3337(_update_code_co_name)
        2    0.000    0.000    0.000    0.000 /usr/lib/python3.12/codeop.py:121(__call__)
        4    0.000    0.000    0.000    0.000 /usr/lib/python3.12/dis.py:671(findlinestarts)
        7    0.000    0.000    0.000    0.000 {built-in method posix.getppid}


<pstats.Stats at 0x7975656be090>
```

3. Google colab T4 GPU -

Execution Time: 5.3176 seconds
Current Memory Usage: 1.31 MB
Peak Memory Usage: 1.32 MB

```
         72 function calls (71 primitive calls) in 5.317 seconds

   Ordered by: internal time
   List reduced from 30 to 10 due to restriction <10>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        4    4.022    1.006    4.022    1.006 {built-in method time.sleep}
        1    0.832    0.832    0.832    0.832 /tmp/ipython-input-
2468726582.py:12(matrix_multiply)
        1    0.461    0.461    5.316    5.316 /tmp/ipython-input-2468726582.py:1(<cell
line: 0>)
        2    0.000    0.000    0.000    0.000 {built-in method builtins.compile}
      3/2    0.000    0.000    5.316    2.658 /usr/local/lib/python3.12/dist-
packages/IPython/core/interactiveshell.py:3512(run_code)
```

```
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        2    0.000    0.000    0.000    0.000 /usr/local/lib/python3.12/dist-
packages/IPython/core/interactiveshell.py:3337(_update_code_co_name)
        2    0.000    0.000    0.000    0.000 /usr/lib/python3.12/codeop.py:121(__call__)
        2    0.000    0.000    5.316    2.658 {built-in method builtins.exec}
        4    0.000    0.000    0.000    0.000 /usr/lib/python3.12/dis.py:671(findlinestarts)


<pstats.Stats at 0x7c3e32945310>
```

4. Google Colab TPU -

Execution Time: 2.8936 seconds
Current Memory Usage: 1.31 MB
Peak Memory Usage: 1.32 MB

```
         68 function calls (67 primitive calls) in 2.893 seconds

   Ordered by: internal time
   List reduced from 30 to 10 due to restriction <10>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        2    2.010    1.005    2.010    1.005 {built-in method time.sleep}
        1    0.479    0.479    2.893    2.893 /tmp/ipython-input-2468726582.py:1(<cell
line: 0>)
        1    0.403    0.403    0.403    0.403 /tmp/ipython-input-
2468726582.py:12(matrix_multiply)
        2    0.000    0.000    0.000    0.000 {built-in method builtins.compile}
      3/2    0.000    0.000    2.893    1.446 /usr/local/lib/python3.12/dist-
packages/IPython/core/interactiveshell.py:3512(run_code)
        1    0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
        2    0.000    0.000    0.000    0.000 /usr/local/lib/python3.12/dist-
packages/IPython/core/interactiveshell.py:3337(_update_code_co_name)
        2    0.000    0.000    0.000    0.000 /usr/lib/python3.12/codeop.py:121(__call__)
        3    0.000    0.000    0.000    0.000 {built-in method posix.getppid}
        4    0.000    0.000    0.000    0.000 /usr/lib/python3.12/dis.py:671(findlinestarts)


<pstats.Stats at 0x7cae6f3a56a0>
```

Table: Performance Comparison Table

Platform	Execution Time (s)	Speedup (vs Colab CPU)	Current Memory (MB)	Peak Memory (MB)
Local (ARM CPU)	1.7134	4.41×	1.33	1.33
Google Colab CPU	7.5553	1.00×	1.31	1.32
Google Colab T4 GPU	5.3176	1.42×	1.31	1.32
Google Colab TPU	2.8936	2.61×	1.31	1.32

Key Observations

1. Local execution on an ARM-based CPU is the fastest among all platforms, outperforming even Google Colab TPU for this serial workload.
2. Execution time trend observed:

   Local CPU < TPU < GPU < Colab CPU

3. Memory usage remains nearly constant (~1.31–1.33 MB) across all platforms because:
   1. The algorithm is strictly serial
   2. No platform-specific parallel libraries are used
   3. Data structures remain identical
4. Function-level profiling (cProfile) shows that:
   1. The majority of execution time is spent inside matrix_multiply()
   2. Python's nested loops are the primary performance bottleneck
   3. Hardware accelerators are not explicitly utilized
5. Despite GPU and TPU showing speedup over Colab CPU, this improvement does not indicate true parallel execution.

Important Note

Even though GPU and TPU platforms show reduced execution time,

the code does not actually use GPU or TPU cores.

The observed speedup is due to:

- Faster underlying hardware
- Optimized runtime environments in Google Colab
- Reduced system overhead

To achieve real GPU/TPU acceleration, **parallel libraries such as NumPy, CUDA, or JAX must be used**.

Final Conclusion

Serial triple-loop matrix multiplication demonstrates significant variation in execution time across different platforms. The local ARM-based CPU achieved the best performance due to lower system overhead and efficient single-core execution. Although Google Colab GPU and TPU runtimes show improved performance compared to Colab CPU, they do not fully utilize their parallel capabilities for pure Python code. This experiment highlights that hardware accelerators alone are insufficient for performance gains without algorithmic parallelization and optimized libraries.